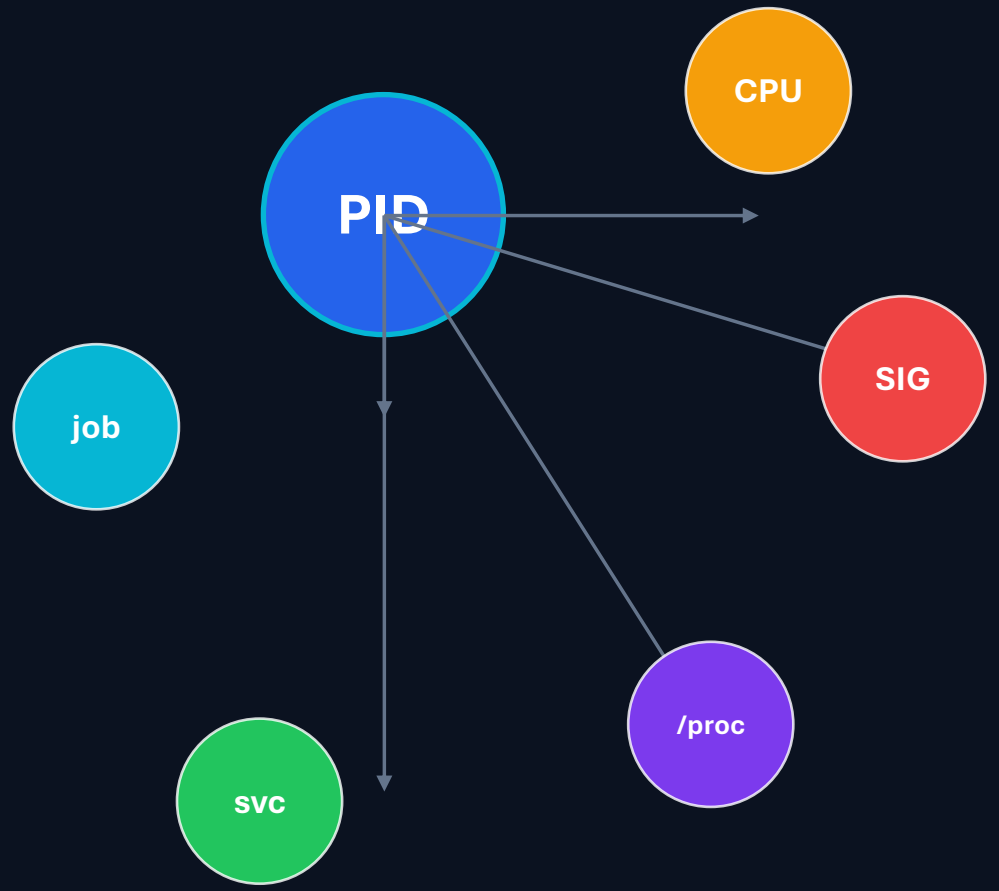
Linux Process Operations Center

A practical project for students to learn Linux processes, signals, CPU troubleshooting, /proc, zombie processes, and systemd service recovery.

ROOT ACCOUNT EDITION · NO sudo

ROCKY LINUX VM



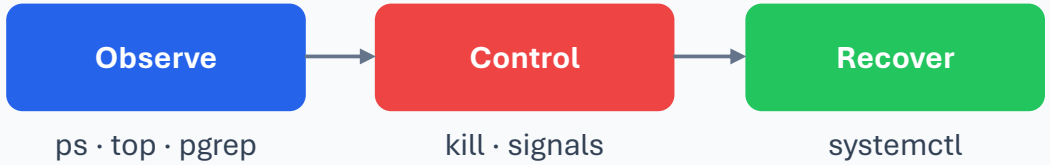
Student builds, breaks, observes, fixes, and validates.

Why this project matters

Students move from memorizing commands to operating a realistic Linux incident.

Scenario

A Nexus Ventures runs a service called NIT Report Generator. It runs continuously, writes logs, sometimes consumes CPU, and can fail. Students act as Linux administrators.



1

Admin muscle memory

Students repeatedly use real operational commands until the workflow becomes natural.

2

Safe failure practice

The lab intentionally creates high CPU, stopped processes, and service failures inside a VM.

3

Evidence-based validation

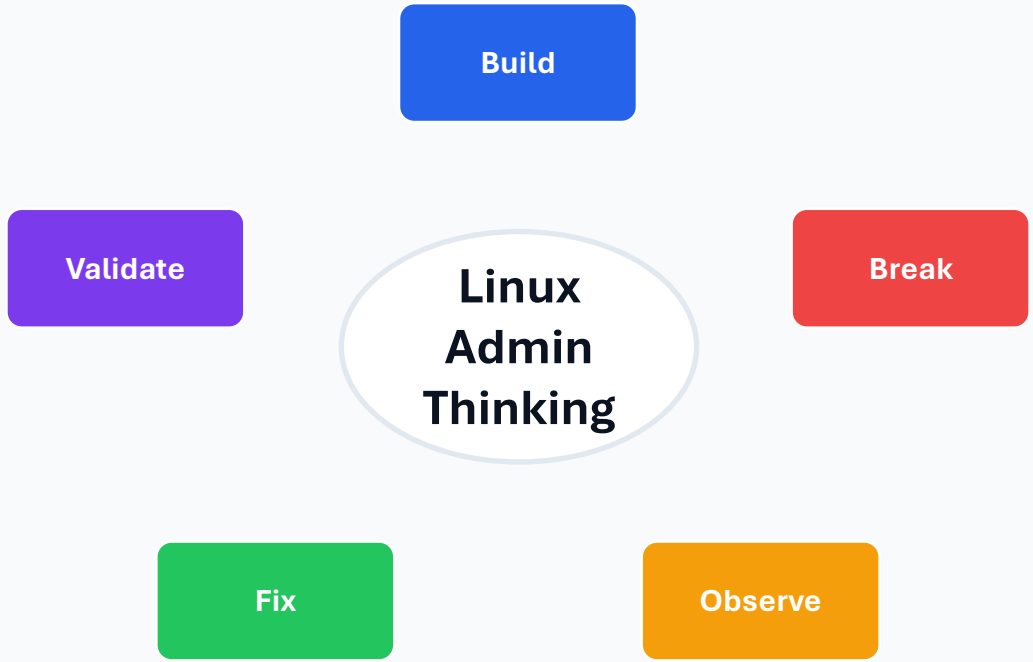
Every phase has commands, expected results, screenshots, and a final validation script.

Learning map

The project links basic process concepts to real troubleshooting actions.

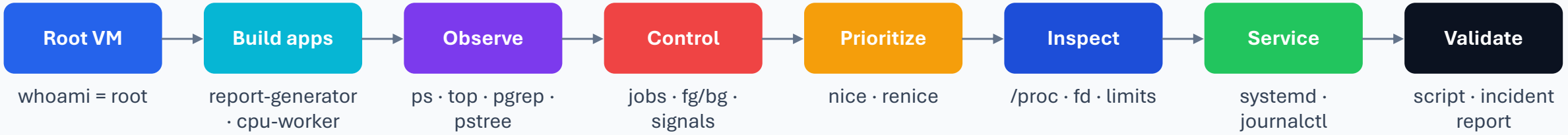
1. **PID / PPID**
   identify ownership and parent-child relationships
2. **Foreground / Background**
   jobs, bg, fg, Ctrl+C, Ctrl+Z
3. **Signals**
   SIGSTOP, SIGCONT, SIGTERM, SIGKILL
4. **Priority**
   nice and renice under load
5. **/proc**
   inspect live kernel process data
6. **systemd**
   run, restart, recover, validate

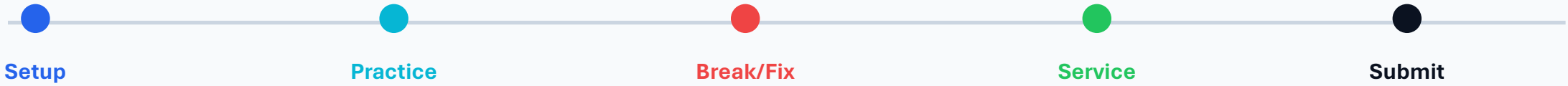
Project learning cycle



Project flow

Full student journey from setup to final validation.

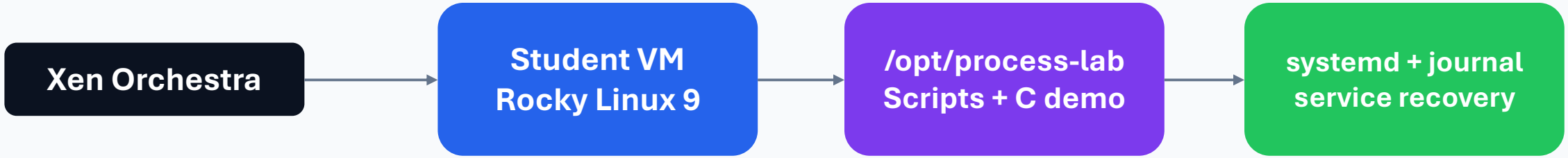




Outcome: students can explain what happened, prove it with commands, fix it safely, and document the incident.

Lab environment: root-account edition

Every student works inside their assigned Xen Orchestra VM using the root account.




```
whoami
id
dnf install -y procps-ng psmisc gcc tree
mkdir -p /opt/process-lab
cd /opt/process-lab
```

✓

No sudo required

All commands assume the user is root. Students validate this before beginning.

!

Isolated lab only

The CPU workload and zombie exercise are safe only inside a student VM.

Phase 1: build the long-running application

Students create a simple service-like script and inspect its live process metadata.

```
# /opt/process-lab/report-generator.sh
LOG_FILE="/tmp/report-generator.log"
while true; do
  echo "$(date): PID $$ is generating a report" >>
"$LOG_FILE"
  sleep 10
done
```

1

Run in foreground

./report-generator.sh keeps the terminal busy until stopped with Ctrl+C.

2

Find the PID

pgrep -af report-generator and ps -o user,pid,ppid,stat,cmd show process details.

3

Validate logging

tail -n 5 /tmp/report-generator.log proves that work is happening every 10 seconds.

Process → Log Evidence

Foreground, background, and shell jobs

Students see the difference between a shell job number and a Linux PID.


```
./report-generator.sh &
jobs -l
fg %1
# Press Ctrl+Z
bg %1
kill %1
```

%

Job number

%1 belongs to the current shell session and is used with jobs, bg, fg, and kill.

PID

Process ID

PID is assigned by the kernel and is visible in ps, top, pgrep, and /proc.

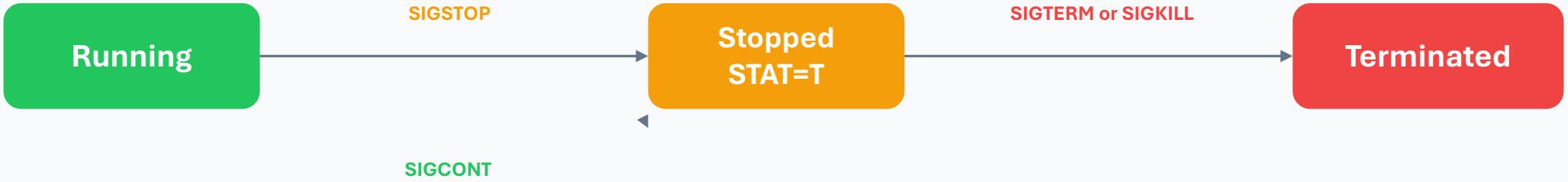
✓

Validation

jobs -l should show the job state and its PID before the process is stopped.

Signals and process lifecycle

Students test stop, continue, graceful termination, and forceful termination.




```
kill -STOP "$REPORT_PID"
ps -o pid,ppid,stat,cmd -p "$REPORT_PID"
kill -CONT "$REPORT_PID"
kill -TERM "$REPORT_PID"
kill -9 "$REPORT_PID"
```

Teach the safe order

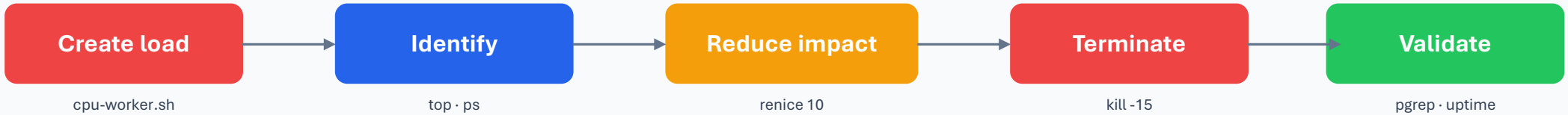
Try SIGTERM first because it lets the application clean up. Use SIGKILL only when needed.

Evidence

Students capture process states, log behavior during SIGSTOP, and final process absence.

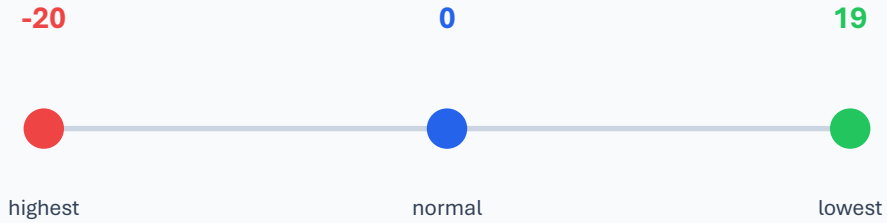
High CPU troubleshooting and priority

Students create a CPU problem, identify it, lower priority, and terminate it safely.




```
./cpu-worker.sh &
CPU_PID=$!
ps -eo pid,ppid,user,stat,%cpu,%mem,ni,cmd --sort=-%cpu | head
renice 10 -p "$CPU_PID"
kill -15 "$CPU_PID"
```

Priority scale



Higher nice number = lower scheduling priority.

Phase 6–7: nohup and /proc investigation

Students prove persistence after logout and inspect process data from the kernel view.


```
nohup ./report-generator.sh > /tmp/report-output.log 2>&1
&
echo $! | tee /tmp/nohup-report.pid
exit
# reconnect
ps -o pid,ppid,stat,etime,cmd -p "$(cat /tmp/nohup-
report.pid)"
```

/

/proc/<PID>/status

Name, State, Pid, PPid, Uid, Threads, memory fields.

fd

/proc/<PID>/fd

Open file descriptors reveal what the process has open.

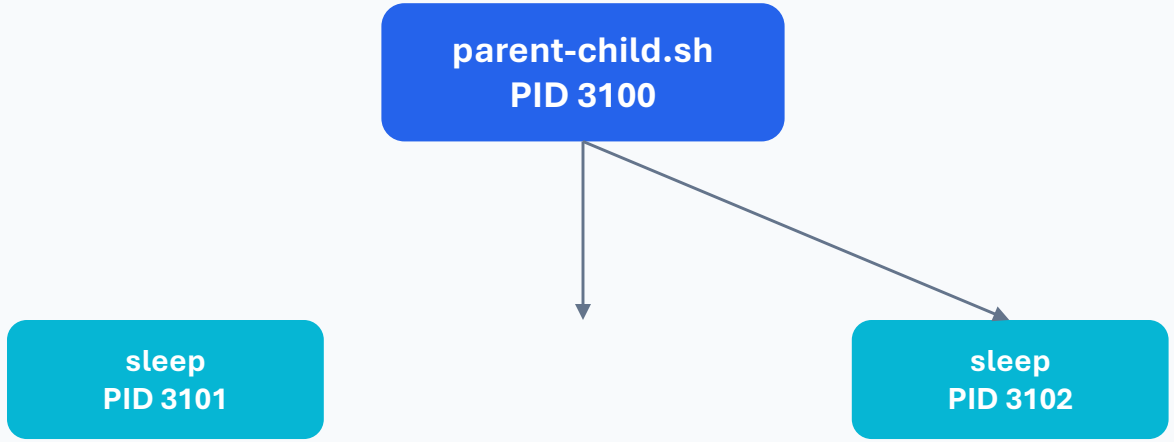
lim

/proc/<PID>/limits

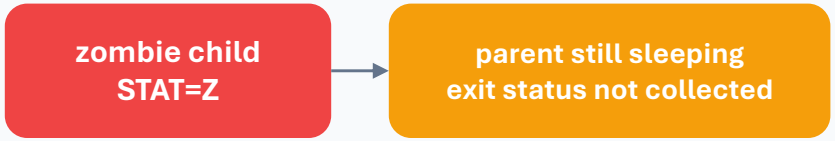
Shows maximum open files, file size, processes, and other limits.

Parent, child, and zombie processes

Students visualize process trees and learn why zombies are handled through the parent.



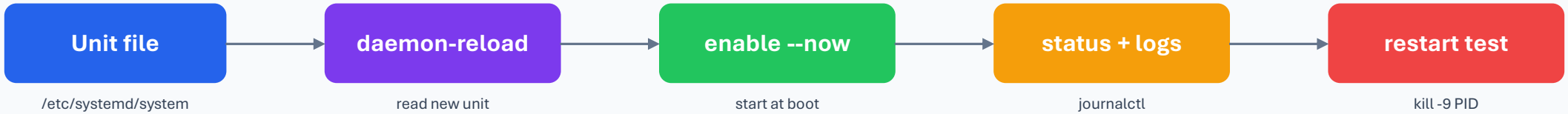

```
pstree -p "$PARENT_PID"
pgrep -P "$PARENT_PID" -a
ps -eo pid,ppid,stat,cmd | awk '$3 ~ /^Z/ {print}'
kill -15 "$ZOMBIE_PARENT"
```



Key idea: a zombie is already terminated. Killing the parent lets the system clean up the zombie entry.

Phase 10–11: systemd service lifecycle

Students turn the script into a managed service that can restart after failure.



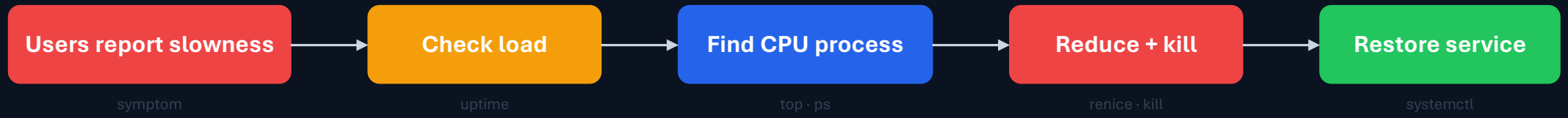

```
[Service]
User=reportsvc
ExecStart=/opt/process-lab/report-generator.sh
Restart=on-failure
RestartSec=5

systemctl enable --now report-generator.service
systemctl show report-generator.service --
property=MainPID
```

1	5s	0
service account	restart delay	sudo needed

Validation: active + enabled + MainPID + owner = reportsvc + new PID after forced failure.

Final challenge: resolve the incident

A realistic mini-incident ties together every process skill.




```
uptime
ps -eo pid,ppid,user,stat,%cpu,%mem,ni,etime,cmd --sort=-
%cpu | head
renice 15 -p "$CPU_INCIDENT_PID"
kill -15 "$CPU_INCIDENT_PID"
systemctl restart report-generator.service
journalctl -u report-generator.service -n 30 --no-pager
```

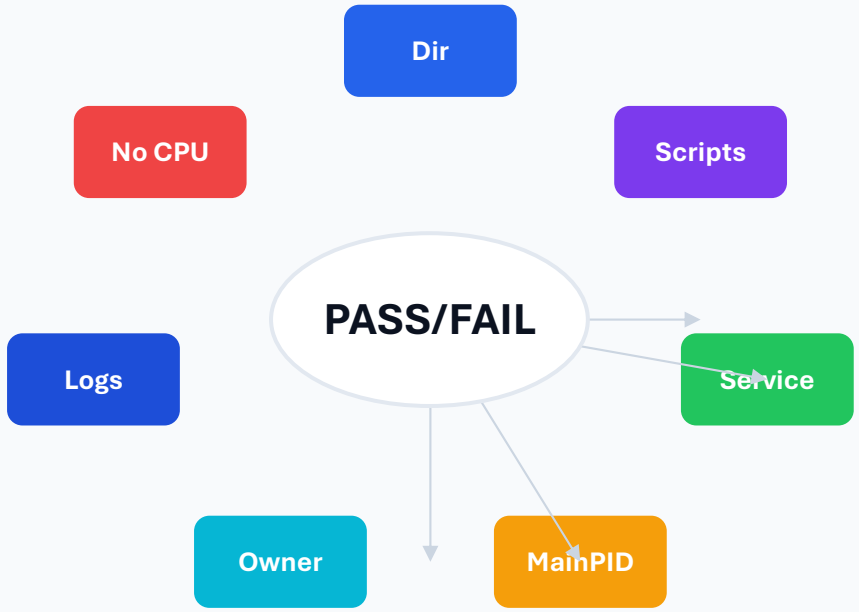
Resolved when:

- ✓ CPU worker gone
- ✓ Service active
- ✓ Logs resumed
- ✓ Load decreasing

Automated validation

The validation script checks files, service status, ownership, logs, and cleanup.

```
chmod +x /opt/process-lab/validate-project.sh
/opt/process-lab/validate-project.sh

# Expected final line:
PROJECT VALIDATION SUCCESSFUL
```



Students should not submit the project until the validation script passes and the incident report is complete.

Student deliverables and grading evidence

Each deliverable must show commands and results, not just a final answer.

1

Process evidence

ps, pgrep, pstree, jobs -l

2

Signal evidence

SIGSTOP, SIGCONT, SIGTERM, SIGKILL

3

Performance evidence

top/ps output, renice result, CPU worker removed

4

Kernel view

/proc/status, fd, limits, environment

5

Service evidence

active, enabled, MainPID, restart after failure

6

Incident report

symptoms, root cause, fix, validation, recommendations

Grading focus: correct execution + evidence + explanation.

Instructor wrap-up

The project turns isolated commands into an operational troubleshooting workflow.

Identify

who owns a process and what state it is in

Control

pause, resume, terminate, and prioritize safely

Explain

parent/child, zombie, /proc, and service lifecycle

Recover

restore a failing service and prove the fix

Final skill: students can troubleshoot a Linux process incident from symptom → evidence → fix → validation.